5.4 Design patterns

Project: Electrical Consumption Monitor — EnergyMonitor **Record:** 3145556 — Software Analysis and Development (ADSO), SENA CIES Regional Huila **Version:** 1.0

> **Update note (10/27/2026).** This document describes the design of version 1.0, when the backend was the `energy-monitor-backend` modular monolith (Spring Modulith). That design evolved: the seven bounded contexts are now **seven microservices** behind the `en-monitor-api-gateway` API gateway (8080) — security 8082, device 8084, measurement 8085, home 8086, notification 8087, alert 8088 and recommendation 8089 —, communicating through internal REST (`/internal/**`) and RabbitMQ events with a transactional outbox, on the same shared MySQL schema (37 tables). The modular backend was retired. The hexagonal layers, contexts and patterns described here live on inside each service; the current architecture, routes, events and configuration are in the technical manual (sections 9 to 13) and the current data model in `06-base-datos`.

1. Purpose

A design pattern is a reusable solution to a problem that appears frequently in software design. Applying a pattern does not consist of copying its structure, but of recognizing **the problem** and choosing the solution that fits best, knowing how to measure its cost as well.

This section documents the patterns used in the "Electrical Consumption Monitor" project, classified according to the two classic catalogs:

- **Creational patterns** — how objects are constructed.
- **Structural patterns** — how objects are composed.
- **Behavioral patterns** — how objects collaborate.

Each pattern is documented with four elements: the problem it solves, how it is applied in this project (with the concrete evidence in the code), what benefit it brings and what cost it introduces.

2. Summary of applied patterns

#	Category	Pattern	Component	Status
1	Structural	**Adapter (Adapter)**	Backend and front-ends	Implemented
2	Structural	**Ports and adapters (Hexagonal)**	Backend	Implemented
3	Structural	**Facade (Facade)**	Backend	Implemented
4	Structural	**Composition and aggregation**	Data model	Implemented
5	Behavioral	**Observer (Observer)**	Backend	Implemented with events
6	Behavioral	**Strategy (Strategy)**	Backend	Implemented
7	Behavioral	**Chain of responsibility**	Backend	Implemented
8	Behavioral	**Filter chain**	Backend and front-ends	Implemented
9	Behavioral	**Command (Command)**	Backend	Implemented
10	Creational	**Factory method (Factory Method)**	Backend	Implemented
11	Creational	**Singleton**	Backend	Implemented by the container
12	Creational	**Builder (Builder)**	Backend	Implemented
13	Creational	**Repository (Repository)**	Backend	Implemented
14	Creational	**Prototype (Prototype)**	Front-ends (instantiation in contextual)	Implemented
15	Behavioral	**Proxy (Proxy)**	Backend	Implemented

#	Category	Pattern	Component	Status
16	Domain	**Value Object (Value Object)**	Backend	Implemented
17	Domain	**Domain Object (Entity)**	Backend	Implemented
18	Domain	**Domain Service (Domain Service)**	Backend	Implemented

3. Structural patterns

3.1 Adapter (Adapter)

Problem. A system must work with an interface it did not design: a database, a mail service, a message broker or the API of another module.

Application in the project. The backend implements interfaces ("ports") in `application/port/out/` and provides "adapter" classes in `adapter/out/` that connect them to the concrete technology.

Port (interface)	Adapter (implementation)	Hidden technology
`PasswordHasherPort`	`BCryptPasswordHasherAdapter`	BCrypt
`PasswordResetTokenHasherPort`	`PepperedHmacResetCodeHasher`	HMAC-SHA256 with pepper
`PasswordResetTokenHasherPort`	`Sha256TokenHasher`	SHA-256
`TokenGeneratorPort`	`SecureTokenGeneratorAdapter`	`SecureRandom`
`IdentifierGeneratorPort`	`SecureIdentifierGeneratorAdapter`	`SecureRandom`
`ResetAttemptLimiterPort`	`InMemoryResetAttemptLimiter`	In-memory structures
`PushSenderPort`	`WebPushSender`	Browser Web Push API
`UserPersistencePort`	`UserPersistenceAdapter`	Spring Data JPA + Hibernate
`HomeApi` (from `home::api`)	`HomeApiImpl`	Internal contract of the homes context
`GoogleIdentityPort`	`GoogleOAuthClientAdapter`	Google API
`ProfileImageFetcherPort`	`GoogleProfileImageFetcher`	Download of the Google image
`PasswordResetDeliveryPort`	`EmailPasswordResetDeliveryAdapter`	SMTP
`EmailVerificationDeliveryPort`	`EmailVerificationDeliveryAdapter`	SMTP

Benefit. The domain contains not a single line of BCrypt, SMTP or SQL code. A test of the `AuthenticateUserService` use case replaces `PasswordHasherPort` with a double and runs without a Spring container.

Cost. It multiplies the number of classes: 64 output ports for 64 contracts. A small module (for example `alert`, with a single table) may turn out to be disproportionate with respect to the benefit.

3.2 Ports and adapters (Hexagonal architecture)

Problem. Coupling the business model to the framework, to the database or to the transport protocol makes it impossible to reuse the model and difficult to verify.

Application in the project. Each module declares two sets of interfaces:

Port type	Location	Responsibility	Example
Input port	`application/port/in`	Declares a business operation that the outside world can request	`CreateHome`, `RegisterMeasurement`
Output port	`application/port/out`	Declares a dependency that the module needs from the outside	`HomePersistencePort`

Verified rule. The domain code (`domain/model`) only imports `java.*`. The `security` use cases do not import Spring either: they are wired with explicit `@Bean` methods, not with `@Service`.

> **Documented exception.** `HomeDeviceService` is the only use case annotated with `@Service` and `@Transactional`. It is a legacy class from an earlier phase that breaks the module rule. It should migrate to the explicit wiring of `DeviceServiceConfiguration` in the next iteration.

Benefit. It allows three things that no other structure allows here: 1. Testing the use cases without a server or a database. 2. Replacing a technology without touching the domain (for example, changing MySQL for PostgreSQL). 3. Verifying the architecture: `ModularityTests` fails if a domain imports something that does not belong to it.

Cost. The number of files grows noticeably: in the `security` module there are 14 domain entities, 23 output ports, 17 input ports and 19 DTOs. The learning curve for a new developer is the price of this structure.

3.3 Facade (Facade)

Problem. A complex subsystem must expose a simple interface for frequent use.

Application in the project. `SecurityLoginFlow` is the facade of the sign-in process: it coordinates five internal operations and returns a single result (`LoginOutcome`).

```
SecurityLoginFlow.login(email, password, ip)
  │
  ├──► AuthenticateUser.authenticate(...)      →  validates credentials, status and verification
  ├──► CreateUserSession.create(...)           →  creates the session
  ├──► RefreshToken.root(...)                  →  generates the first refresh generation
  └──► JwtTokenIssuer.issue(...)               →  signs the access token
       │
       └──► LoginOutcome { accessToken, refreshToken, expiresIn, account }
```

The `AuthController` controller does not know any detail of this process: it calls a single method.

Other facades in the project:

Class	Simplifies
`SecurityFilterChainConfiguration`	The Spring Security security DSL, which would otherwise occupy a method with more than forty lines
`SecurityCorsConfiguration`	The construction of `CorsConfigurationSource`, with validations that throw `IllegalStateException` on unsafe configurations
`HomeDeviceService`	A single class that implements six use case interfaces of the devices module

Benefit. The authentication process can change completely (for example, by adding multi-factor authentication) by modifying a single class.

Cost. The facade can turn into a "God object" that concentrates responsibilities. `SecurityLoginFlow` is kept small and stateless precisely to avoid this.

3.4 Composition and aggregation

Problem. When modeling relationships between objects, a choice must be made between "strong ownership" (the part dies with the whole) and "weak dependency" (both can exist separately).

Application in the project. The choice was made explicitly for each relationship of the data model:

Relationship	Type	Justification
`Home 1 *-- 1 HomeThresholds`	**Composition**	Thresholds do not exist without a home; if the home is deleted, its thresholds disappear
`User 1 *-- 1 UserConfiguration`	**Composition**	The configuration is an attribute of the account, not an entity with a life of its own
`UserSession 1 *-- 1..* RefreshToken`	**Composition**	The session owns its generations; revoking the session invalidates all of them

Relationship	Type	Justification
`Home "*" o-- "1" HomeType`	**Aggregation**	The catalog is shared: a home type serves many and does not belong to any
`Device "*" o-- "1" ApplianceType`	**Aggregation**	The appliance catalog is shared by all devices
`Home 1 -- "0..*" UserHome`	**Association**	Membership is preserved even if the home is deleted (soft delete)
`UserHome` with composite PK	**Composite key**	Membership is identified by the user–home pair, not by an identifier of its own

Benefit. The choice is reflected in the schema (foreign keys `NOT NULL`, uniqueness in `home_thresholds.home_id`) and does not depend on the designer's intention.

Cost. Choosing the cardinality badly forces costly migrations. That is why the rule was documented in the class diagram.

4. Behavioral patterns

4.1 Observer (Observer)

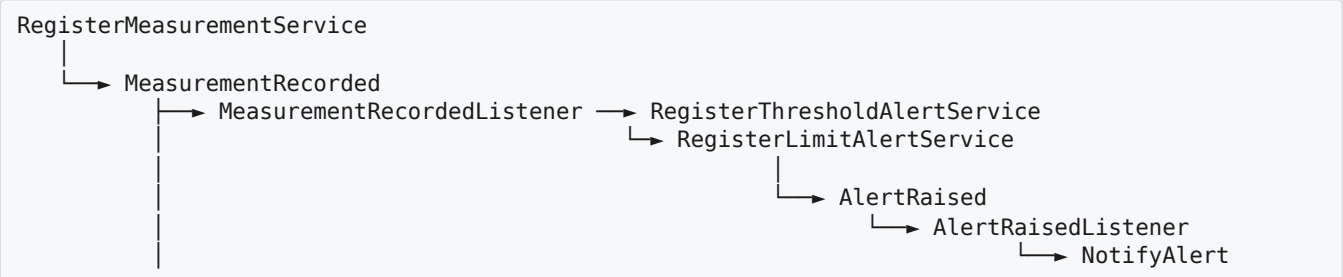
Problem. When an event occurs, several recipients must be notified without the emitter knowing them or depending on them.

Application in the project. It is the central pattern of the reactive flow of the system.

Participants in the measurement chain:

Pattern role	Class	Module
Subject	`MeasurementEventAdapter` (publishes `MeasurementRecorded`)	`measurement`
Observer	`MeasurementRecordedListener` (`@EventListener`)	`alert`
Secondary subject	`AlertEventAdapter` (publishes `AlertRaised`)	`alert`
Observer	`AlertRaisedListener` (`@Async`)	`notification`

Full chain:



Pattern variant — named events. The implementation uses named events (`ApplicationEventPublisher` and `@EventListener`) instead of the observer list of the canonical pattern. The difference is relevant:

Aspect	Canonical Observer	Project implementation
Subscription	The subject maintains the list	The subject only publishes
Coupling	The subject knows its observers	None: whoever publishes ignores who listens
Consequence	Adding an observer forces modifying the subject	Adding an observer does not touch the subject
Loss	An observer failure affects the emitter	An observer failure does not prevent delivery

> **Decision documented in the code.** The `MeasurementRecordedListener` class includes the comment: *"the class diagram's* `AlertObserver`*"*. That is: the class diagram describes the Observer pattern, and the code materializes it with in-process events, which is the variant with the least coupling.

Recommendations (implemented). The `recommendation` module does not observe measurements: `RecommendationScheduler` runs `EvaluateRecommendationsService` every hour, which reads consumption through outbound ports, applies the saving rules and, for each new recommendation, publishes `RecommendationCreated` through `RecommendationEventAdapter`; `notification` listens to it with `RecommendationCreatedListener`.

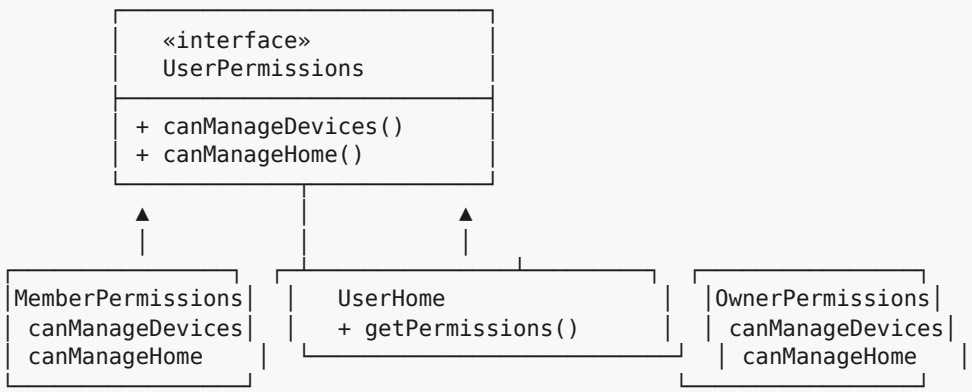
Benefit. The `measurement` module does not know that alerts exist. Another observer can be added without modifying a single line of the measurements module.

Cost. In-process events are **not persistent**: if the process restarts between the publishing and the listening, the event is lost. For the current system this is acceptable (measurements are resent every 60 seconds), but a flow that requires delivery guarantees needs a message queue.

4.2 Strategy (Strategy)

Problem. A behavior must vary according to the object type, without filling the class with `if`/`switch` branches or forcing the caller to know all the variants.

Application in the project. Authorization within a home varies according to the role (`OWNER` or `MEMBER`).



Evidence in the code. The three classes exist in `home/domain/model`, and `UserHome.getPermissions()` returns the implementation that corresponds to its `role`. `DeviceExceptionHandler` and `HomeExceptionHandler` throw `NotHomeOwnerException` when the owner strategy does not allow the operation.

> **Important correction.** The note in the delivered class diagram states that these three classes "do not yet exist in the Java code" and that real authorization is resolved through a generic RBAC chain. **That note is outdated:** the three classes are implemented. The diagram should be corrected before delivery (see `05.3-class-model.md`, section 10).

Complementary design — dual role system. The system has **two** authorization mechanisms that should not be confused:

Mechanism	Scope	Resolution	Example
Per-home role (`Role`)	A user within a specific home	`user_home.role` → `UserHome.getPermissions()`	Only the owner can link devices or expel members
Global RBAC (`CheckPermission`)	Functional permissions of the system	Live query of the `system_role` → `user_system_role` → `system_role_permission` → `permission` chain	Permissions such as `METER.READ` or `ALERT.READ`

Benefit. Adding a third role (for example, `INVITADO`) consists of creating a class that implements `UserPermissions` and assigning it from `UserHome.getPermissions()`. No other class in the system changes.

Cost. The number of classes grows with the number of variants. With two variants, the benefit is smaller than the cost; with five or more, the strategy is clearly justified.

4.3 Chain of responsibility (Chain of Responsibility)

Problem. A request must travel through a sequence of handlers, and each one decides whether to handle it or pass it to the next.

Application in the project. The access authorization to the MQTT broker in `BrokerAuthController.authorize(username, topic, access)` walks through the access levels that the broker requests:

```
switch (request.access()) {
    case 1  -> /* read only     */ break;
    case 2  -> /* write only    */ break;
    case 3  -> /* read and write */ break;
    case 4  -> /* subscribe     */ break;
    default -> /* deny          */ break;
}
```

Benefit. Each permission level is validated with a different rule; adding a level does not force modifying the logic of the previous ones.

Cost. The control flow becomes implicit: to know what a level validates you have to read the complete `switch`.

4.4 Filter chain (Filter Chain)

Problem. A request must pass through a sequence of checks before reaching the resource.

Application in the project. In the backend, `SecurityFilterChainConfiguration` defines a single Spring Security chain:

Order	Component	Function
1	`BrokerAuthSecurityConfiguration` (`@Order(1)`)	Protects `/internal/mqtt/**` by restricting to the authorized network
2	`SecurityCorsConfiguration`	Applies the CORS policy and responds to the `OPTIONS` *preflight*
3	`BearerTokenAuthenticationFilter` (from Spring Security)	Validates the `RS256` JWT and sets the security context
4	`JwtAuthenticationEntryPoint`	Responds `401` with the `ApiError` format
5	`JwtAccessDeniedHandler`	Responds `403` with the `ApiError` format
6	`AuthorizationFilter`	Applies the authorization rules per endpoint

In the front-ends, the same idea appears in the HTTP client interceptors:

Layer	Component	Function
Request	`httpClient` — request interceptor	Adds `Authorization: Bearer`, skips public endpoints and renews the token if it expired
Response	`httpClient` — response interceptor	On `401` tries to renew once and retries the request

Benefit. Each filter has a single responsibility and can be tested separately.

Cost. The order of the filters is implicit; changing it can break security without any test detecting it.

4.5 Command (Command)

Problem. An operation needs to carry input data and output results without coupling the interface to the domain model.

Application in the project. Each input port exposes a method that receives an immutable *record* and returns another.

Component	Example
Command	`RegisterUserCommand`, `CreateHomeCommand`, `RegisterMeasurementCommand`
Query	`CheckPermissionQuery`
Result	`AuthenticationResult`, `HomeResult`, `AlertResult`, `RefreshSessionResult`

Explicit difference between command and query:

Type	Verb	Effect	Example
Command	Imperative: `register`, `create`, `update`, `delete`	Modifies the state	`CreateHomeCommand`
Query	Interrogative: `find`, `list`, `get`, `check`	Only reads	`CheckPermissionQuery`

Benefit. Command records are immutable and can be constructed in any layer. Use cases are tested by constructing the command directly, without building HTTP DTOs.

Cost. It multiplies the types: for a single use case there are often four files (port, command, result and service).

5. Creational patterns

5.1 Factory method (Factory Method)

Problem. A class must create objects without knowing the concrete classes it is going to instantiate, or it must guarantee that its invariants are respected at the moment of construction.

Application in the project. The aggregates offer static factories that guarantee the validity of the created instance:

Aggregate	Factories	Guaranteed invariant
`Alert`	`threshold()`, `limit()`, `connectivity()`, `device()`	A threshold alert has a consumption level and a measurement; a connectivity one does not
`RefreshToken`	`root()`, `childOf()`	The first generation has no parent; the following ones do, and they inherit the family
`Device`	`create()`, `relinked()`, `edited()`, `withApiKey()`	A device always has a unique code and an integration key
`DeviceStatusLog`	`create()`	The log has a status and a last connection
`DeviceHome`	`of()`	The link belongs to a valid device and a valid home
`LoginErrorLog`	`userNotFound()`	A failed attempt with an unknown user is logged without a user identifier
`AuditLog`	`login()`	The audited action complies with the expected format
`Email`, `PasswordHash`	`of()`	The value is valid according to its format

Benefit. A client that wants a limit alert **cannot** build it with inconsistent values: the only path is the correct factory. The invariant does not depend on the discipline of the programmer.

Cost. Each new type of alert requires adding a factory. With ten types, adding becomes costly; for four types it is manageable.

5.2 Singleton

Problem. A class must have a single instance shared by the whole application.

Application in the project. The Spring container manages the lifecycle; all beans are singletons by default. The cases that explicitly require it are declared:

Bean	Class	Reason
`clock()` (`@Primary`)	`Clock.systemUTC()`	Single source of time; it enables tests with a fixed clock
`securityClock()`, `mailDeliveryClock()`	`Clock`	Independent clocks per module
`jwtEncoder()`, `jwtDecoder()`, `jwtKeyId()`	Nimbus	The RSA key pair is read only once from the file system
`passwordResetExecutor()`	`ThreadPoolExecutor`	A single group of bounded threads for password recovery
`mqttClientFactory()`	`MqttPahoClientFactory`	One connection to the broker, not one per message

Benefit. The single clock is what makes it possible to write deterministic tests: if time were read from the system, no token expiration test would be reproducible.

Cost. Global state in singleton beans complicates concurrent tests and can introduce race conditions if a bean accumulates mutable state.

5.3 Builder (Builder)

Problem. An object must be constructed with many optional parameters, and the signature with all of them would be illegible.

Application in the project. The pattern manifests in two forms:

Form	Where	Example
Records as builders	Backend	`RegisterUserCommand(email, password, person, profileImage, ipAddress, zone)` — the compiler requires the mandatory fields and the order is explicit
Template strings	Backend	`NotifyAlert.sendMail(...)` builds the body with text blocks and `.formatted(...)`

Benefit. With records, the compiler guarantees that no mandatory field is forgotten and that no argument is passed in the wrong position.

Cost. Records with many fields are still awkward to build in tests, where helper builders are used (`UseCaseFixtures`, `RefreshTokenFixtures`).

5.4 Repository (Repository)

Problem. The application needs to query and persist objects without knowing the details of storage.

Application in the project. The project uses the pattern at two levels:

Level	Technology	Quantity
OUTPUT PORT (domain)	Interfaces `*PersistencePort`	64 output ports
ADAPTER (infrastructure)	`*Repository extends JpaRepository`	30 interfaces

Complete example of the path of a query:

```
AlertController.getAlerts(homeId, status)
   │
   ▼  ListAlertsService  (use case)
AlertListAlertsPort  ──►  ListAlertsService
   │                              │
   │                              ▼  output port
   │                    AlertPersistencePort
   │                              │
   │                              ▼  adapter
   │                    AlertPersistenceAdapter
   │                              │
```

```
   │                              ▼  Spring Data repository
   │                   AlertRepository  ──►  Hibernate  ──►  table `alert`
   ▼
AlertResponse  (JSON DTO)
```

Benefit. The use case is tested with a double of `AlertPersistencePort`; the real test against MySQL is done at a different layer.

Cost. For very simple tables, the port level is unnecessary. It is the pattern that suffers the cost of hexagonal architecture the most.

5.5 Prototype (Prototype) — instantiation in contextual

Problem. An object must be created from an existing template and then modified.

Application in the project. Both front-ends use React DOM's `createPortal` to render the account modal over `document.body` without unmounting the rest of the tree:

```
Sidebar (inside MainLayout)
   │
   └──► createPortal(<Account/>, document.body)
          └──► Account modal on top of the whole application
```

Mobile variant. The state of the side menu is cloned on each opening by animating a copy of the style through `Animated.timing` with `useNativeDriver: true`, instead of rebuilding the component.

Benefit. It avoids recreating the complete structure of the modal and guarantees that it overlays any content, even if the container has `overflow: hidden`.

Cost. Portals escape the visual flow of the React tree: events and inherited styles must be resolved explicitly.

6. Additional patterns of the project

6.1 Dynamic proxy

Problem. A class needs to incorporate cross-cutting behavior —in this case, a transaction— without its code mentioning it.

Application in the project. `SecurityApplicationWiring` builds a programmatic `ProxyFactory` with a `TransactionInterceptor` instead of annotating the use cases with `@Transactional`:

```
private <T> T transactional(Class<T> type, T target, T port) {
    ProxyFactory proxyFactory = new ProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.setInterfaces(port);   // only the input port interface
    proxyFactory.addAdvice(transactionInterceptor);
    return (T) proxyFactory.getProxy();
}
```

Relevant detail. The proxy implements **only the input port interface**. This means that a call from another point of the system to the concrete implementation **does not** go through the transaction. It is an architectural security barrier, not an oversight.

Benefit. The transactional boundary is explicit and tested: `SecurityTransactionBoundaryTest` verifies with `AopUtils.isAopProxy` which use cases are transactional and `SecurityTransactionRollbackTest` verifies the behavior on failure.

Cost. The correction is invisible to whoever reads the code: a use case shows nothing that indicates where its transaction starts and ends.

6.2 Value Object (Value Object)

Problem. Certain data has no identity: they are values. Treating them as loose strings allows a malformed email address to reach the database.

Application in the project.

Value object	Invariant	How it is guaranteed
`Email`	Valid format, not empty	Static factory `Email.of()`
`PasswordHash`	Not empty, produced by a hasher	Static factory `PasswordHash.of()`
`Preconditions`	Invariant rules of each aggregate	`final` class with package visibility in each module
`Measurement`	Immutable magnitudes after the record	Without setters

Guaranteed properties: equality by value, immutability and interchangeability.

Benefit. `User.changeEmail(newEmail: Email)` cannot receive an arbitrary string. The validation is at construction, not at use.

Cost. A value object cannot have identity, so it is not persisted separately. For data that is sometimes a value and sometimes an identity (an email address that is also a key), the decision requires judgment.

6.3 Domain Object (Entity)

Problem. Some data has identity and a lifecycle; it must guarantee its invariants throughout that lifecycle.

Application in the project. `User`, `Home`, `Device`, `Alert` and `RefreshToken` are domain objects: their behavior lives in the class.

Object	Invariant protected by its own behavior
`User`	`canAuthenticate()` is only true if it is `ACTIVE` and verified
`User`	`incrementFailedLoginAttempts()` never leaves the counter negative
`UserSession`	`isActive(now)` is false if the session is revoked or closed
`RefreshToken`	`isUsable(now)` is false if it is rotated, revoked or expired
`Home`	`addUser(u, role)` validates that the user is not already linked

Benefit. The invariant is maintained without every caller remembering to validate it.

Cost. The object accumulates responsibilities. If too much behavior is modeled in the domain without control, the class becomes difficult to test; the balance is set by the principle "the domain knows nothing about infrastructure".

6.4 Domain Service (Domain Service)

Problem. A behavior that spans several aggregates does not naturally belong to any of them.

Application in the project. `RecommendationRules` is the only class in `domain/service/`. It analyzes measurements from all the devices of a home and returns suggestions: it does not belong to `Measurement` (which only knows a single reading), nor to `Device` (which does not see the whole set), nor to `Recommendation` (which is the result).

Property	Value
Dependencies	None
Input/output	None
Clock	Received as a parameter (`Instant`, `ZoneId`)

Benefit. The rules are pure and deterministic: the same input always produces the same output, in any time zone and without the need for a Spring container.

Cost. A domain service without dependencies is, structurally, a set of functions. The advantage appears when the rules grow; with few rules, a function in the use case would suffice.

7. Patterns in the front-ends

The two front-ends replicate the same set of presentation patterns, which guarantees that the experience is homogeneous.

#	Pattern	Application on the web	Application on mobile
1	**Design tokens**	CSS variables in `:root`, reassigned per theme	Token system in `constants/` and palettes in `theme/`
2	**Dynamic theme**	`ThemeProvider` writes CSS variables in the root element	`createThemeStyles` rebuilds the sheets when the palette changes
3	**Composition over inheritance**	`children` in `Input`, `Header`, `Card`	`children` in `Header`, `Modal`, `Alert`
4	**Variants**	`variant` in `Button`, `Card`, `Input`, `Badge`	`variant` in `Button`, `Card`, `Input`, `Modal`, `Alert`
5	**State lifting**	`Context` + `useSyncExternalStore` for the session	`Context` + `useSyncExternalStore` for the session
6	**Context composition**	`useOutletContext()` shares the resource with the child route	Nested contexts in `_layout.tsx`
7	**Single data layer**	`services/` with a single HTTP client	`services/` with a single HTTP client
8	**State machine**	Steps of the Bluetooth linking wizard	Steps of the linking wizard
9	**Declarative states**	`loading` / `error` / `empty` / `content`	`loading` / `error` / `empty` / `content`
10	**Single flight queue**	Token renewal serialized with Web Locks	Token renewal serialized with a module promise

7.1 Highlighted detail — Mobile theme system

The most interesting case of interface design is the theme in React Native. React Native has no CSS variables, so the web and mobile solved the same problem in different ways:

Approach	Solution
Web	CSS variables written in `:root`; the browser recomputes everything by itself
Mobile	`createThemeStyles` wraps `StyleSheet.create` in a `Proxy` that memoizes the sheet per palette and rebuilds it when it changes

Additional problem documented in the code. With the React compiler enabled, the render of each component is memoized according to its reactive dependencies. The theme palette is not among them, so a screen can reuse its cached JSX and keep the previous styles.

Adopted solution. The 13 mobile routes render with `key={currentTheme.id}`. The key forces the remount of the component and guarantees a full repaint. It is a conscious decision, documented in `ThemeContext.tsx`, that resolves a real limitation of the combination between the React compiler and the theme tokens.

8. Decisions on applying or not applying a pattern

Not all problems require a pattern. The decisions of **not** applying one are recorded:

Situation	Decision	Justification
Global state in the front-ends	**Context, not Redux nor Zustand**	The global state is reduced (session, theme, homes, notifications). Redux would add ceremony and boilerplate without benefit. `zustand` (web) and `@reduxjs/toolkit` with `react-redux` (mobile) appear in the `package.json` files, but the source code does not import them

Situation	Decision	Justification
Charts	**Specialized library, not an own pattern**	Recharts and `react-native-chart-kit` solve the problem; writing an own charting engine adds no value
Form validation	**Zod, not an own validator**	The declarative schema covers the case and produces types
Data caching in the front-ends	**No cache, with polling**	The data volume is low and on-demand loading is sufficient. A cache would require invalidation, which is more complex than the problem it solves
Pagination	**Not implemented**	No system table currently grows beyond a few dozen rows

9. Traceability of the patterns with the requirements

Pattern	Requirement or constraint it satisfies
Ports and adapters	RNF-MAN (maintainability) — use cases are tested without infrastructure
Executable module boundaries	RNF-MAN — the architecture is verified automatically
Observer	RF-ALE (automatic alerts) — decouples measurement from alert
Strategy	RF-HOM-05 (user permissions) — authorization by role
Factory method	RF-ALE — alert invariants at the moment of creating them
Filter chain	RNF-SEG (security) — authentication and authorization controls
Repository	RNF-MAN — decouples the domain from the SQL schema
Value object	RNF-SEG (security) — validation at construction, not at use
Design tokens	RNF-USA (usability) — visual coherence and consistency between platforms
Context composition	RNF-USA — predictable navigation flow
Declarative states	RNF-USA — clear messages on error and empty state

10. Related artifacts

Artifact	Location
Class model with the patterns	`05.3-class-model.md`
Class diagram (PlantUML)	`assets/diagramas/Diagrama_clases.wsd`
Architectural model	`05.1-architectural-model.md`
Verifiable architectural rules	`05.1-architectural-model.md`, section 8
Tests that verify the boundaries	`energy-monitor-backend/src/test/java/com/energymonitor/ModularityTests.java` (repository retired on 10/28/2026; valid for version 1.0, today each service is an independent repository and deployment, so the boundary between contexts is the service itself)